

Guía de ejecución reproducible - SmartClean ETL

Esta guía permite que el docente o cualquier integrante clone el repositorio en otra computadora y obtenga los mismos resultados del proyecto.

1. Requisitos previos

- Windows 10/11 de 64 bits, Linux o macOS.
- Docker Desktop con Docker Compose v2.
- En Windows, WSL 2 y virtualización habilitados.
- Git.
- Al menos 8 GB de RAM disponibles.
- Puertos libres: 1435, 5432 y 8000.

Verificación rápida:

```
git --version
docker --version
docker compose version
docker run hello-world
```

2. Clonar el repositorio

```
git clone https://github.com/sofi-hobi/sales-data-migration-etl.git
cd sales-data-migration-etl
```

Asegúrese de trabajar con la rama principal actualizada:

```
git switch main
git pull --ff-only origin main
```

3. Crear el archivo de configuración

Git Bash o Linux/macOS

```
cp .env.example .env
```

PowerShell

```
Copy-Item .env.example .env
```

CMD

```
copy .env.example .env
```

No es necesario modificar el archivo `.env` para la prueba estándar. El archivo `.env` no debe subirse a GitHub; solamente se versiona `.env.example`.

4. Ejecutar desde cero

Abra Docker Desktop y espere hasta que indique que el motor está activo. Desde la raíz del proyecto ejecute:

```
docker compose down -v --remove-orphans
docker compose up -d --build
```

La primera ejecución puede tardar varios minutos porque descarga y construye las imágenes de SQL Server, PostgreSQL, Python y la aplicación web.

5. Comprobar el estado

```
docker compose ps -a
```

Estado esperado:

Servicio	Estado normal	Significado
sqlserver	Up (healthy)	La base de origen está creada y tiene al menos 1.000 clientes.
postgres	Up (healthy)	La base de destino está disponible.
etl_pipeline	Exited (0)	El ETL terminó correctamente. No es un error.
webapp	Up (healthy) o Up	La interfaz está disponible.

Si etl_pipeline aparece como Exited (1), revise sus registros:

```
docker compose logs --tail=200 etl_pipeline
```

6. Abrir la aplicación

Abra en el navegador:

```
http://localhost:8000
```

La interfaz muestra el total procesado y una muestra de hasta 100 filas por tabla. La muestra visual no limita el número de registros procesados.

7. Resultados esperados

Métrica	SQL Server - origen	PostgreSQL - destino
Clientes	1.200	1.098 clientes maestros
Productos	60	55 productos maestros
Facturas	1.206	1.206
Detalles	2.412	2.412
Total facturado	120.011,78	120.011,78

La reducción de clientes y productos se debe a la consolidación de duplicados. Las facturas, los detalles y el total monetario deben conservarse.

8. Validar SQL Server

```
docker compose exec sqlserver bash -lc '/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "$MSSQL_SA_PASSWORD'
```

Validaciones principales:

- 1.200 clientes en dbo.ClienteOrigen.
- 1.206 facturas.
- 2.412 detalles.
- 0 facturas sin cliente.
- 0 detalles sin factura.
- 0 detalles sin producto.

Conteo directo de clientes:

```
docker compose exec sqlserver bash -lc '/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P "$MSSQL_SA_PASSWORD'
```

9. Validar PostgreSQL

```
docker compose exec postgres psql -U etl_user -d etl_destino -c "
SELECT 'clientes' AS tabla, COUNT(*) AS cantidad FROM clientes
UNION ALL SELECT 'productos', COUNT(*) FROM productos
UNION ALL SELECT 'facturas', COUNT(*) FROM facturas
UNION ALL SELECT 'detalles', COUNT(*) FROM detalles;
"
```

Comprobar que no queden clientes repetidos por documento o correo:

```

docker compose exec postgres psql -U etl_user -d etl_destino -c "
SELECT documento, COUNT(*)
FROM clientes
WHERE documento IS NOT NULL
GROUP BY documento
HAVING COUNT(*) > 1;

SELECT correo, COUNT(*)
FROM clientes
WHERE correo IS NOT NULL
GROUP BY correo
HAVING COUNT(*) > 1;
"

```

Ambas consultas deben devolver cero filas.

10. Revisar auditoría y consolidación

```

docker compose exec postgres psql -U etl_user -d etl_destino -c "
SELECT COUNT(*) AS mapeos FROM cliente_origen_mapeo;
SELECT COUNT(*) AS grupos_consolidados FROM auditoria_consolidacion;
SELECT COUNT(*) AS errores_calidad FROM etl_transformacion_errores;
SELECT * FROM etl_validacion_resultado ORDER BY id_validacion DESC LIMIT 10;
"

```

Estas tablas permiten comprobar qué registros de origen se relacionaron con cada cliente maestro y qué inconsistencias fueron detectadas.

11. Probar la reejecución

Con las bases levantadas, ejecute de nuevo:

```
docker compose run --rm etl_pipeline
```

Después vuelva a revisar los conteos. Deben mantenerse en:

- 1.098 clientes maestros.
- 55 productos maestros.
- 1.206 facturas.
- 2.412 detalles.

Esto demuestra que la carga es idempotente y no duplica datos al repetirse.

12. Ver registros de diagnóstico

```

docker compose logs --tail=200 sqlserver
docker compose logs --tail=200 postgres
docker compose logs --tail=200 etl_pipeline
docker compose logs --tail=200 webapp

```

Guardar todos los registros en un archivo:

```
docker compose logs --no-color > logs_proyecto.txt
```

13. Detener o reiniciar el proyecto

Detener sin eliminar los datos:

```
docker compose down
```

Eliminar contenedores y volúmenes para una prueba completamente limpia:

```
docker compose down -v --remove-orphans
```

Volver a construir:

```
docker compose up -d --build
```

14. Problemas frecuentes

El puerto ya está ocupado

Cierre el servicio que utiliza el puerto o modifique el puerto externo en `docker-compose.yml`. Los puertos predeterminados son 1435, 5432 y 8000.

Docker no puede conectarse al motor

Abra Docker Desktop y espere hasta que muestre **Engine running**. En Windows también puede ejecutar:

```
wsl --shutdown
```

Después abra Docker Desktop otra vez.

La web permanece en “Cargando datos”

Revise:

```
docker compose ps -a
docker compose logs --tail=200 etl_pipeline
docker compose logs --tail=200 webapp
```

La web depende de que el ETL termine con código 0.

Se modificaron scripts de creación, pero no cambian las tablas

Los scripts de PostgreSQL se ejecutan al crear el volumen. Realice una prueba limpia:

```
docker compose down -v --remove-orphans
docker compose up -d --build
```

15. Prueba final antes de entregar

Para comprobar que el repositorio no depende de archivos locales, clónelo en otra carpeta y repita el proceso:

```
cd ..
git clone https://github.com/sofi-hobi/sales-data-migration-etl.git prueba-profesor
cd prueba-profesor
cp .env.example .env
docker compose down -v --remove-orphans
docker compose up -d --build
docker compose ps -a
```

Si la copia recién clonada muestra los resultados esperados en `http://localhost:8000`, el proyecto está listo para ser ejecutado por el docente en otra computadora.